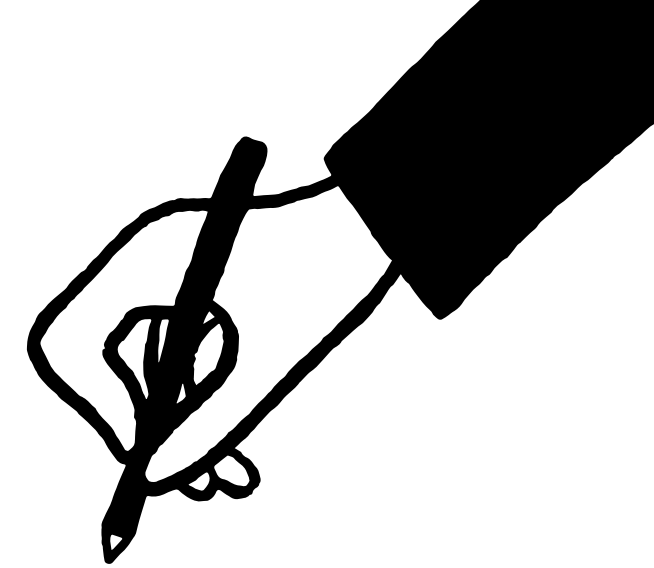


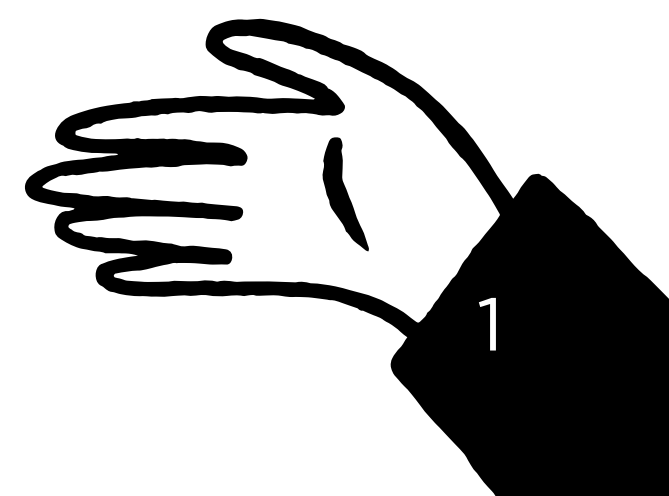


Git Cooperativo: como não bagunçar o código dos outros

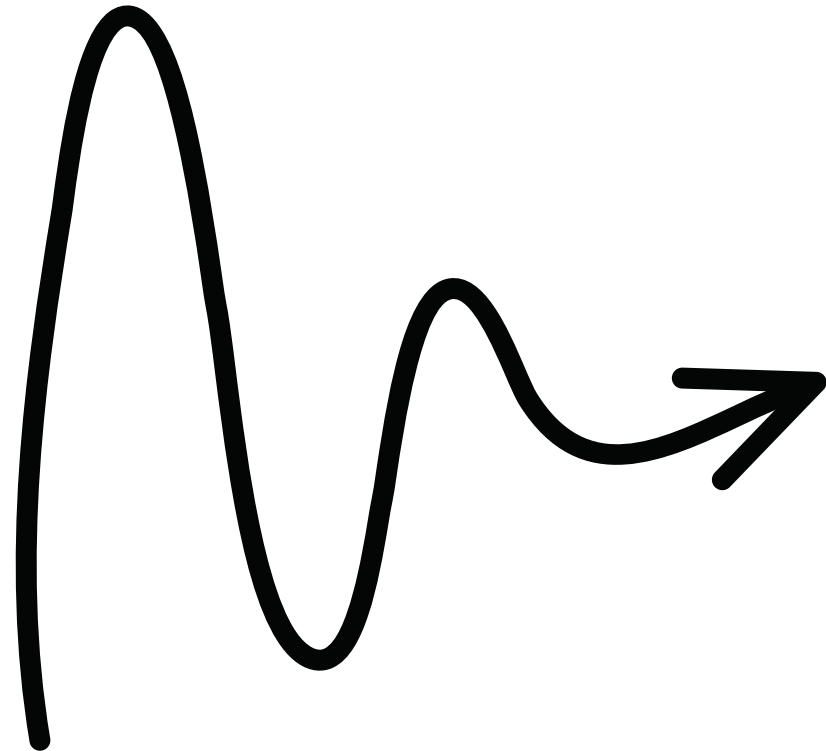
**Uma oficina prática para
colaborar em projetos sem dor de
cabeça.**

Palestrantes: Carolina Borges, Felipe
Selau e Rafael Ramos

Data: 21/10/25

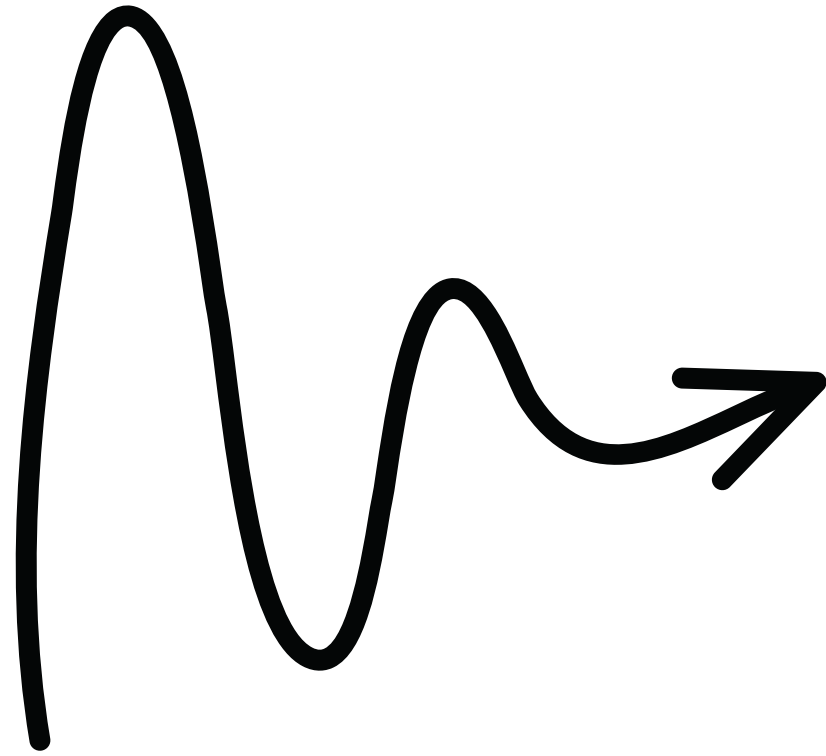


Programação



1. **Checklist de preparação do ambiente**
2. **Por que utilizar Git?**
3. **Conceitos fundamentais**
4. **Importando o projeto com git clone**
5. **Trabalhando em uma branch**
6. **Enviando nossas alterações**
7. **Guardando um trabalho inacabado**
8. **Abrindo um pull request e solicitando code review**
9. **Boas práticas**
10. **Mão na massa**

O que vamos aprender?



- **Sincronizando com a equipe:** como clonar e atualizar um projeto da equipe.
- **Trabalho individual seguro:** como usar branches, commits e o stash para trabalhar sem interferir no código dos colegas.
- **Colaboração e resolução de problemas:** como enviar suas alterações, criar um Pull Request, fazer revisão de código e resolver conflitos.

Checklist de preparação do ambiente

Para essa oficina, você precisa:

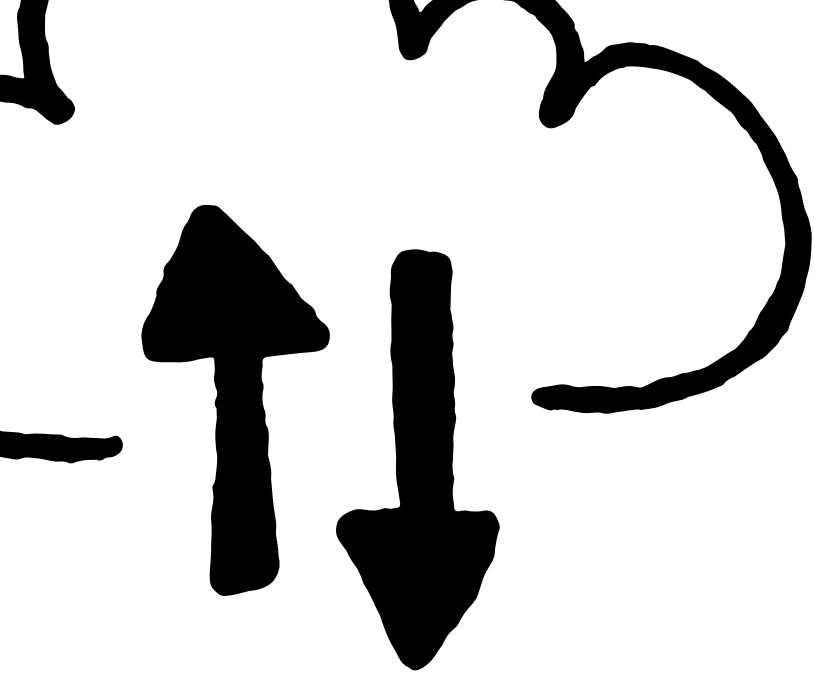
- Ter o Git instalado no computador
- Ter uma conta no Github
- Ter um editor de código instalado (VSCode)
- Abrir um terminal, pode ser o do VSCode ou o Git Bash (terminal do Git)

Link do projeto que vamos usar para a prática:

<https://github.com/RafaelR4mos/if-semana-academica-git>

if-semana-academica-git:

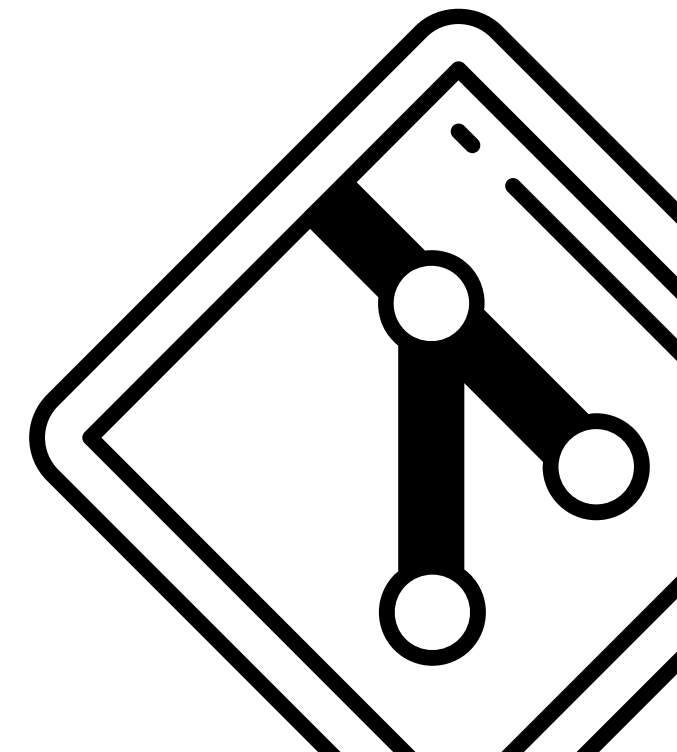


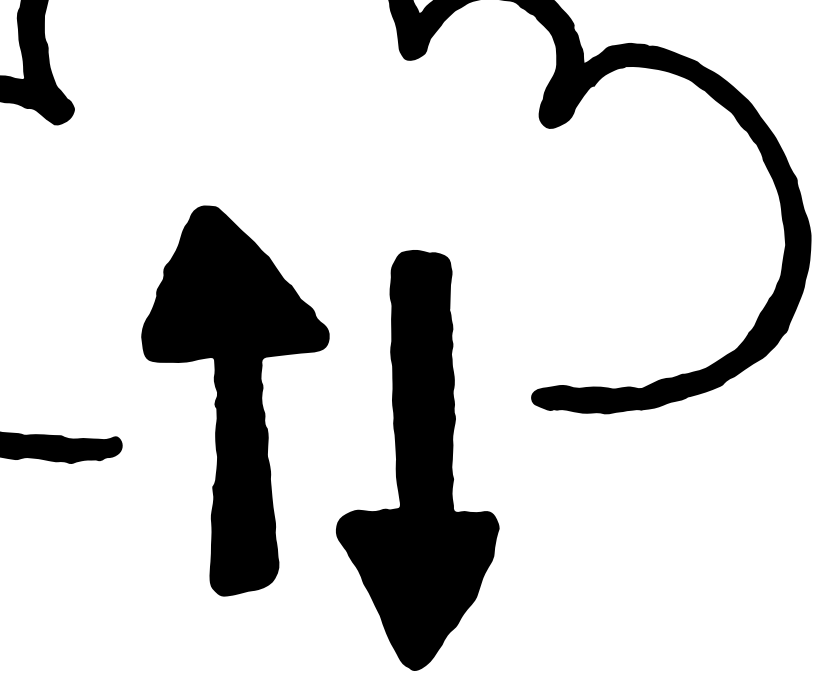


Por que utilizar o Git?

Ao invés de ter vários arquivos no PC como *“index-versao-final.html”* ou *“index-versão-finalizada-mesmo-agora-vai.html”*, você tira uma “foto” (commit) do seu código, e envia pro Git, que vai ter armazenado todo o histórico de “fotos” (commits) que foram tiradas.

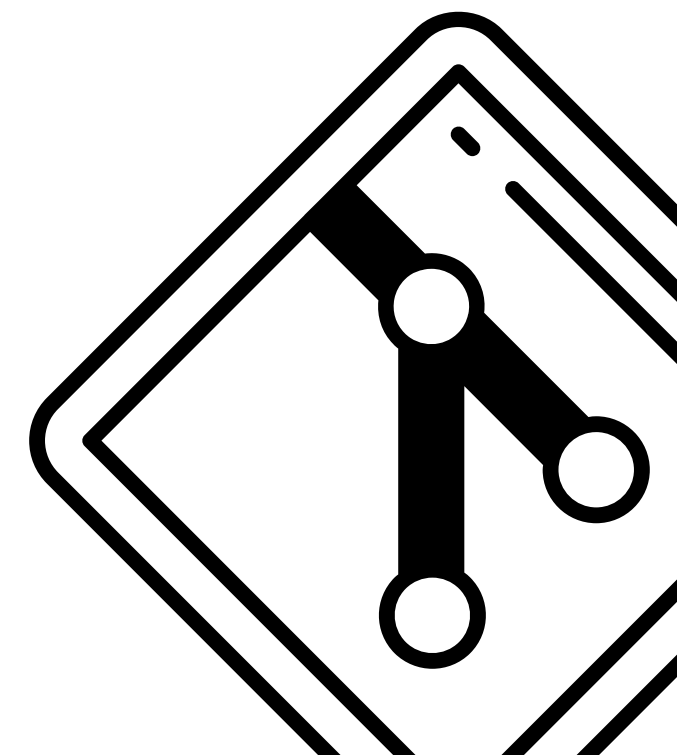
- Isso é chamado de **versionamento**

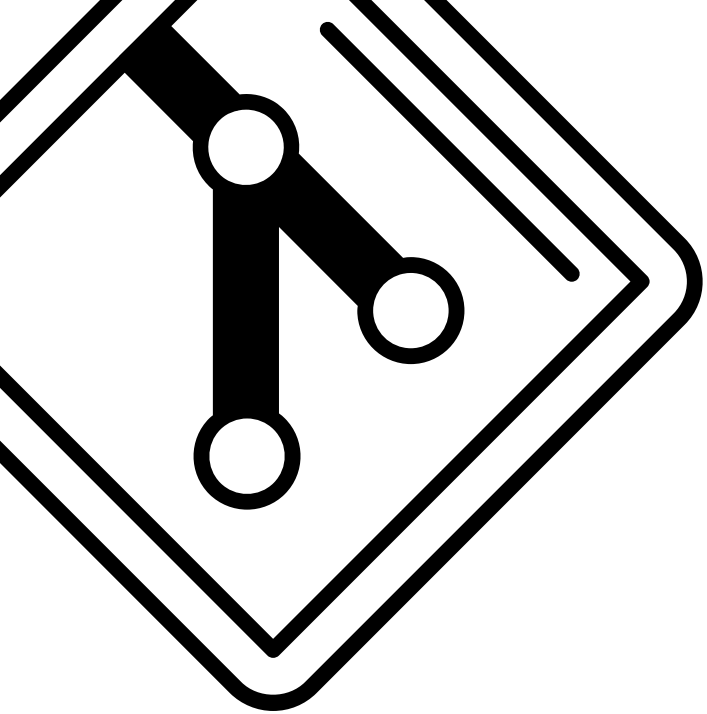




Por que utilizar o Git?

- Histórico de alterações
- Garante integridade do código
- Trabalho paralelo entre os colegas
- Saber **controle de versão (versionamento)** é indispensável no mercado de tecnologia

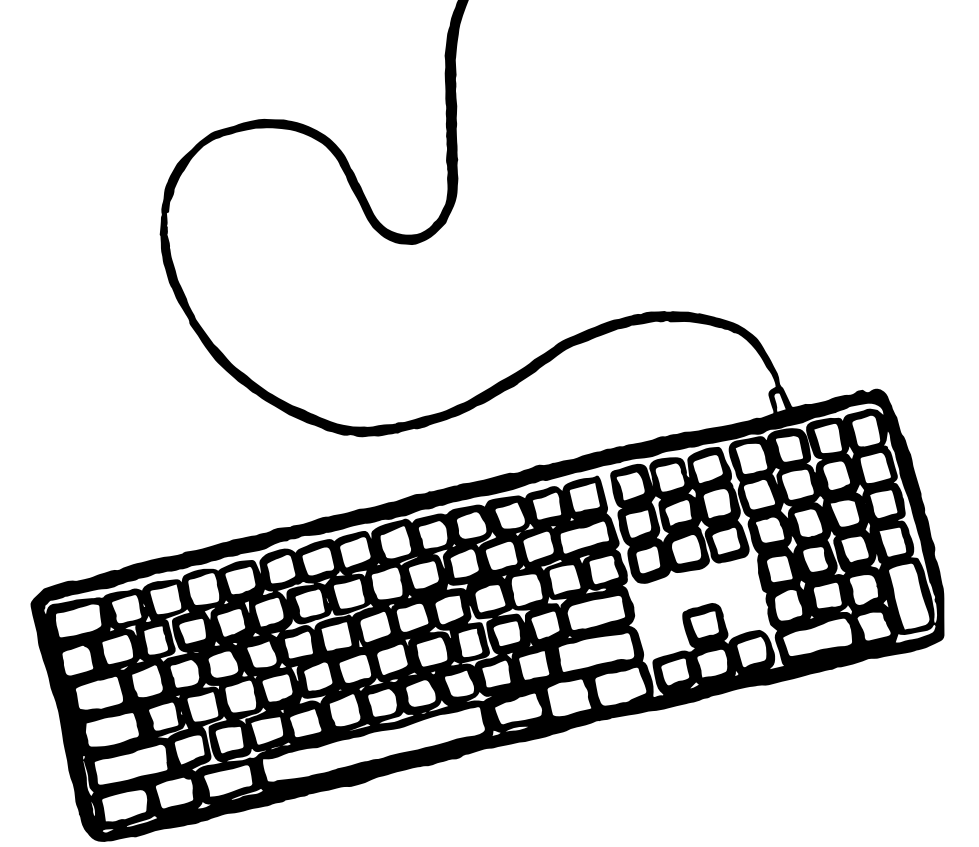




Conceitos fundamentais

- **Repositório** → é onde fica localizado o projeto e o histórico de alterações dele
- **Branch** → ramificação do projeto para realizar qualquer mudança sem interferir na branch principal
- **Commit** → uma atualização do projeto com as alterações que queremos deixar visíveis
- **Merge** → junção do código de uma branch com o código de outra branch
- **Pull Request** → uma solicitação de merge, que requer um code review das alterações feitas
- **Code Review** → trabalho de revisão de um pull request para identificar erros ou melhorias. É feito por um ou mais colegas que não sejam os autores do pull request.
- **Conflitos** → quando duas branches diferentes tiveram alterações nas exatas mesmas linhas de código, e é feita uma tentativa de merge dessas branches

Importando o projeto com git clone



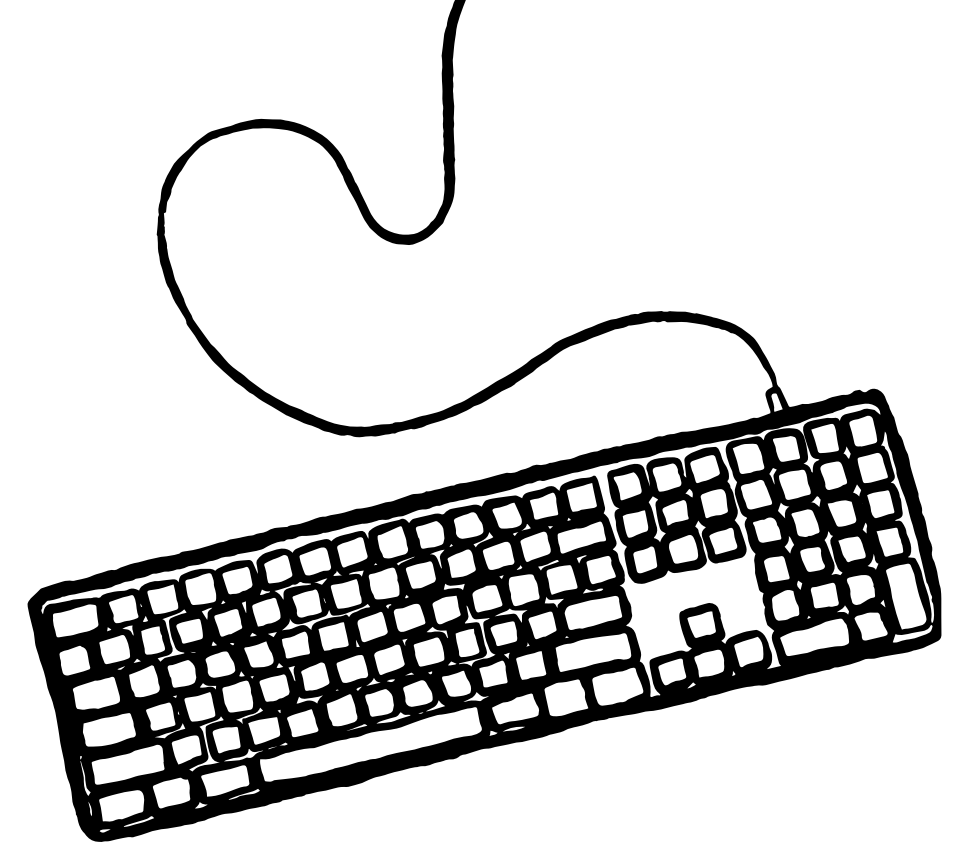
No seu terminal rode os comandos abaixo, um por vez:

```
git clone https://github.com/RafaelR4mos/if-semana-academica-git.git  
cd semana-academica-git  
code .
```


Trabalhando em uma branch

No seu terminal rode o comando abaixo:

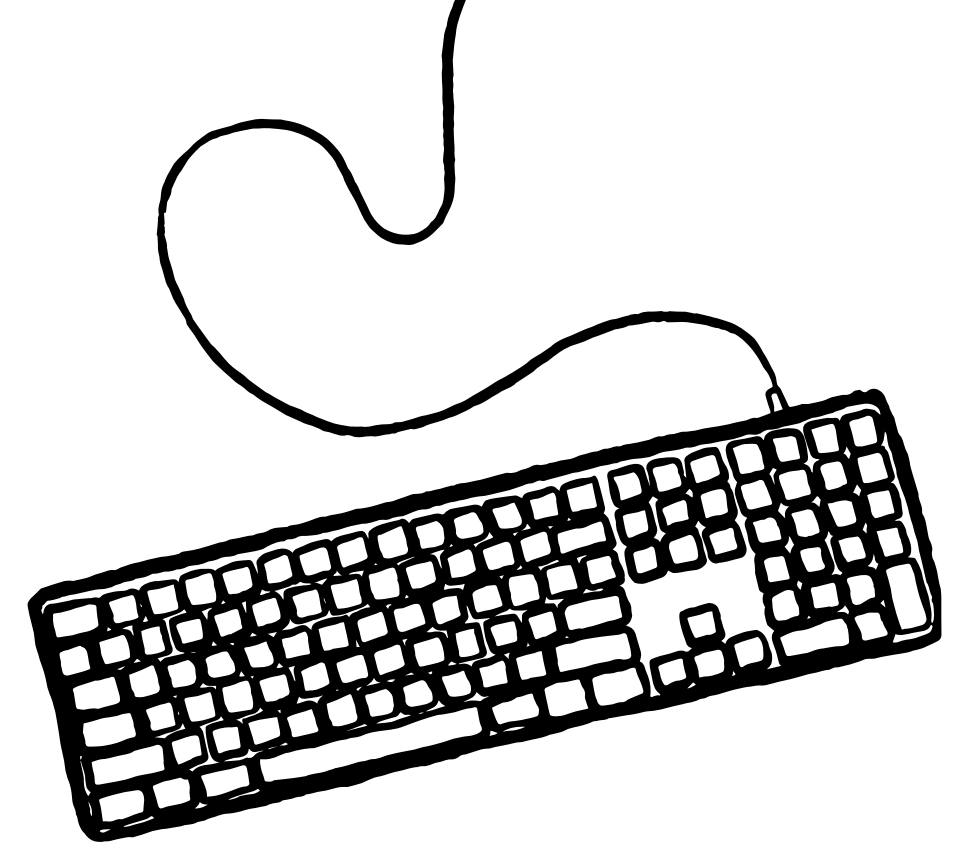
```
git checkout -b feature/<nome-feat>
```



Enviando nossas alterações

Na sua branch, edite `src/index.html` e adicione seu nome na linha:

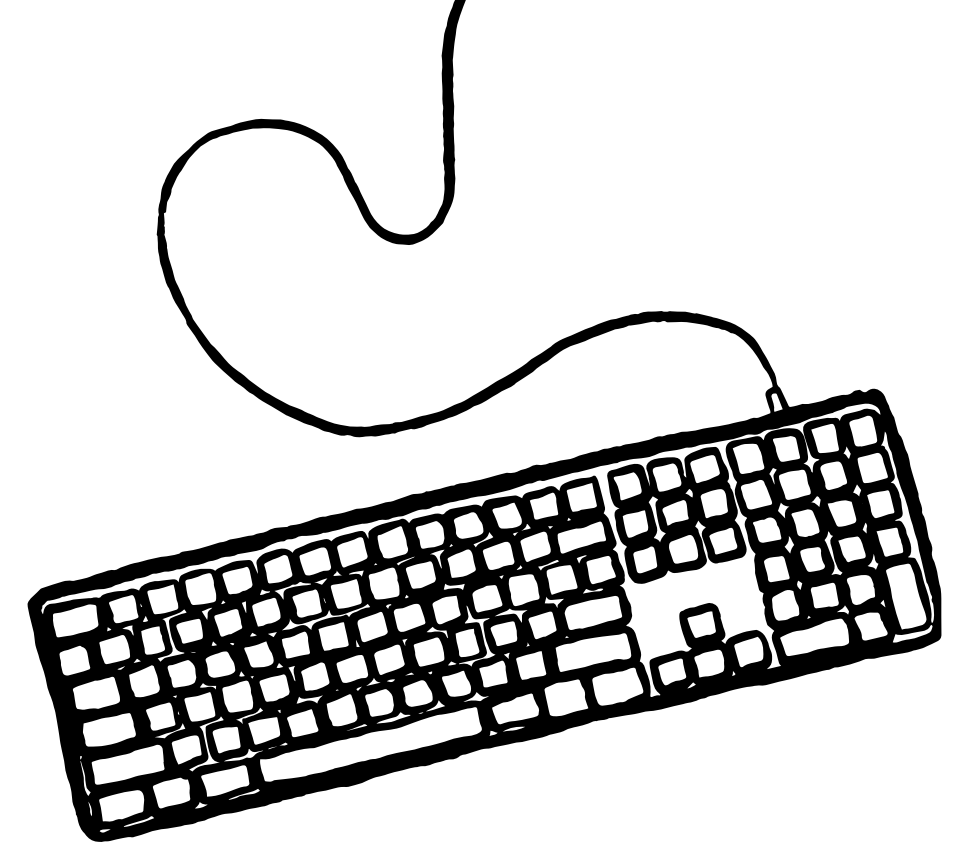
```
<li>SEU NOME</li>
```



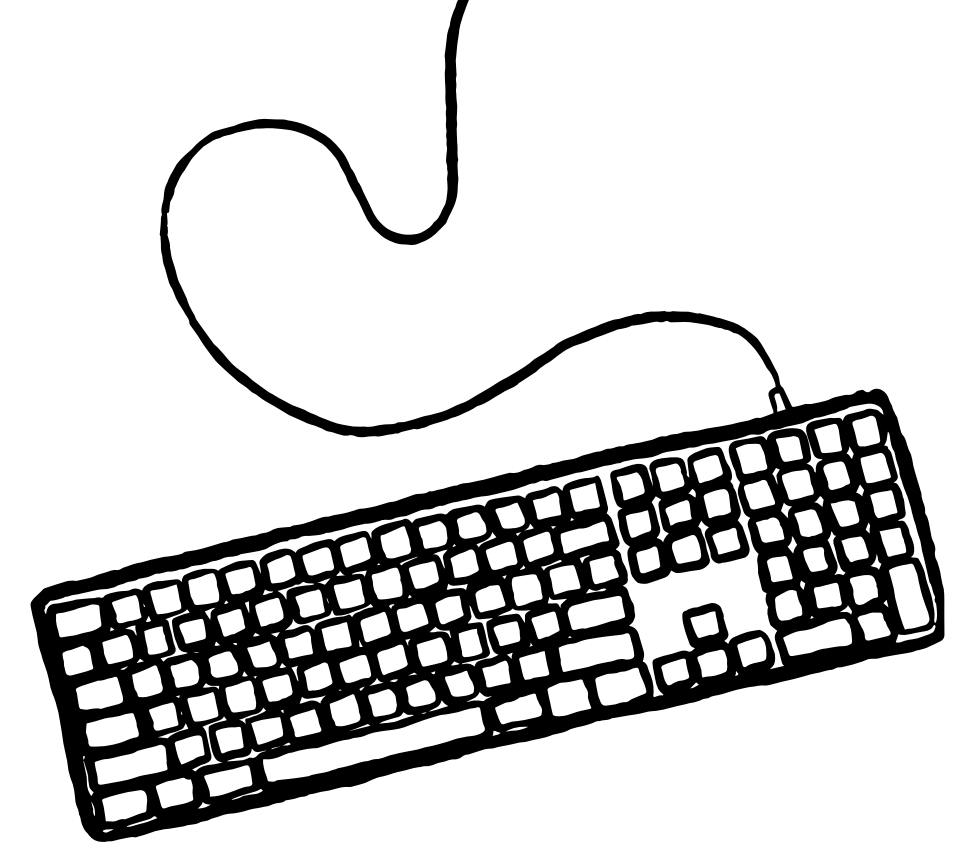
Enviando nossas alterações

Salve o arquivo com CTRL + S e rode o comando abaixo:

```
git status
```



Enviando nossas alterações



Deve aparecer uma mensagem como essa:

```
On branch feature/minhaBranch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

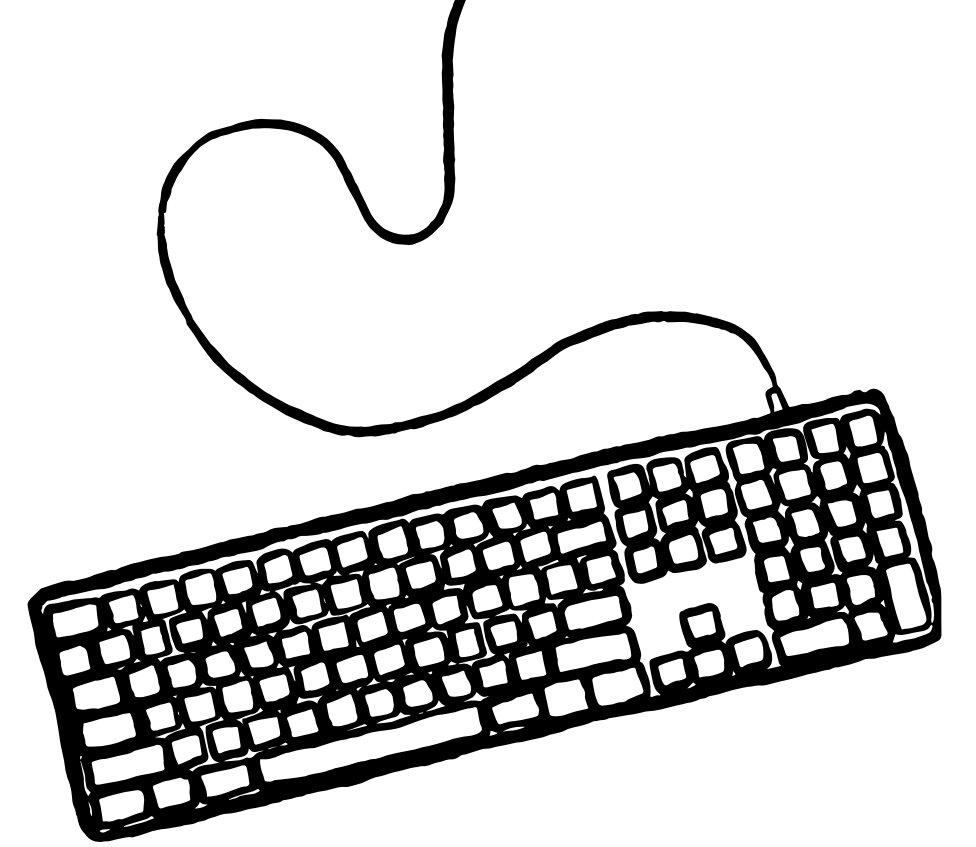
Enviando nossas alterações

Para contornar essa situação, rode um dos comandos abaixo:

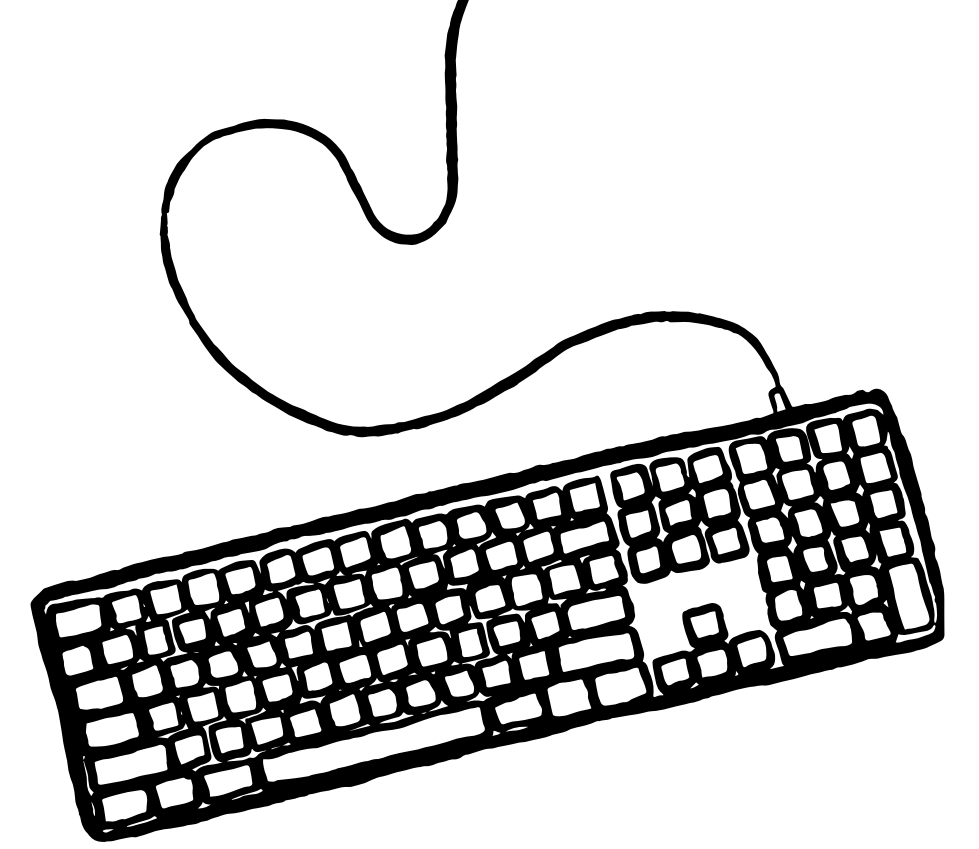
```
git add .
```

ou

```
git add src/index.html
```



Enviando nossas alterações



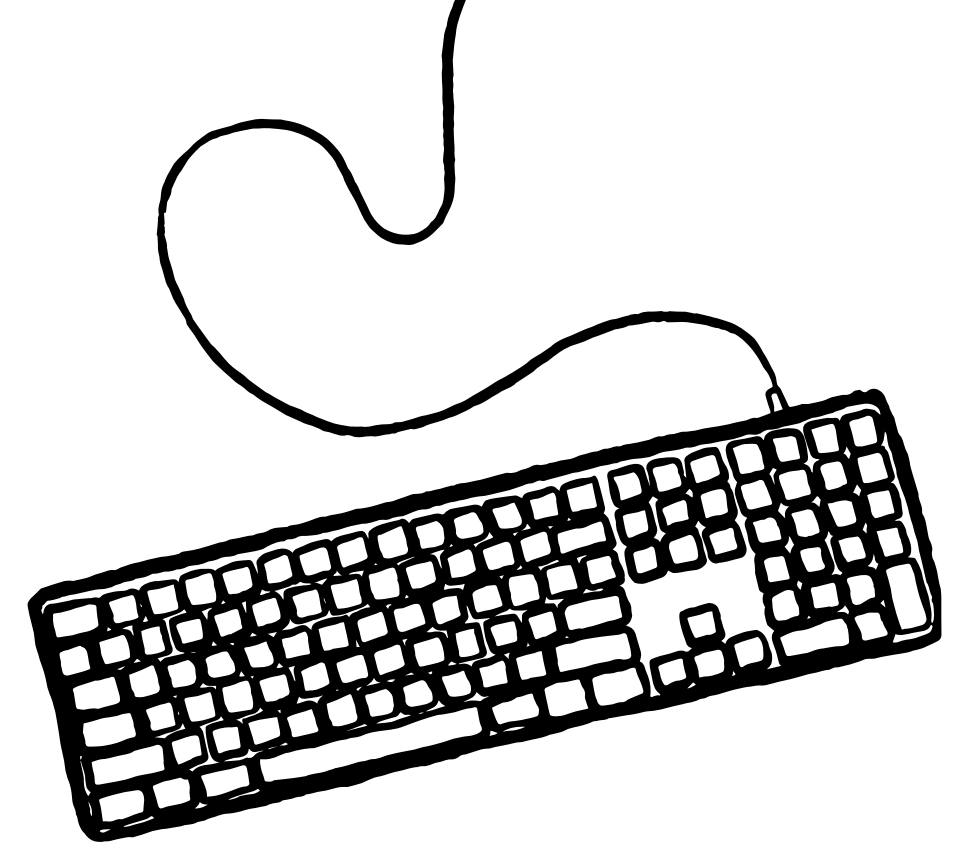
Agora o arquivo estará pronto para commit:

```
On branch feature/minhaBranch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   src/index.html
```

Enviando nossas alterações

Rode o comando abaixo e verifique se mudou algo no github

```
git commit -m "feat: adicionando meu nome"
```

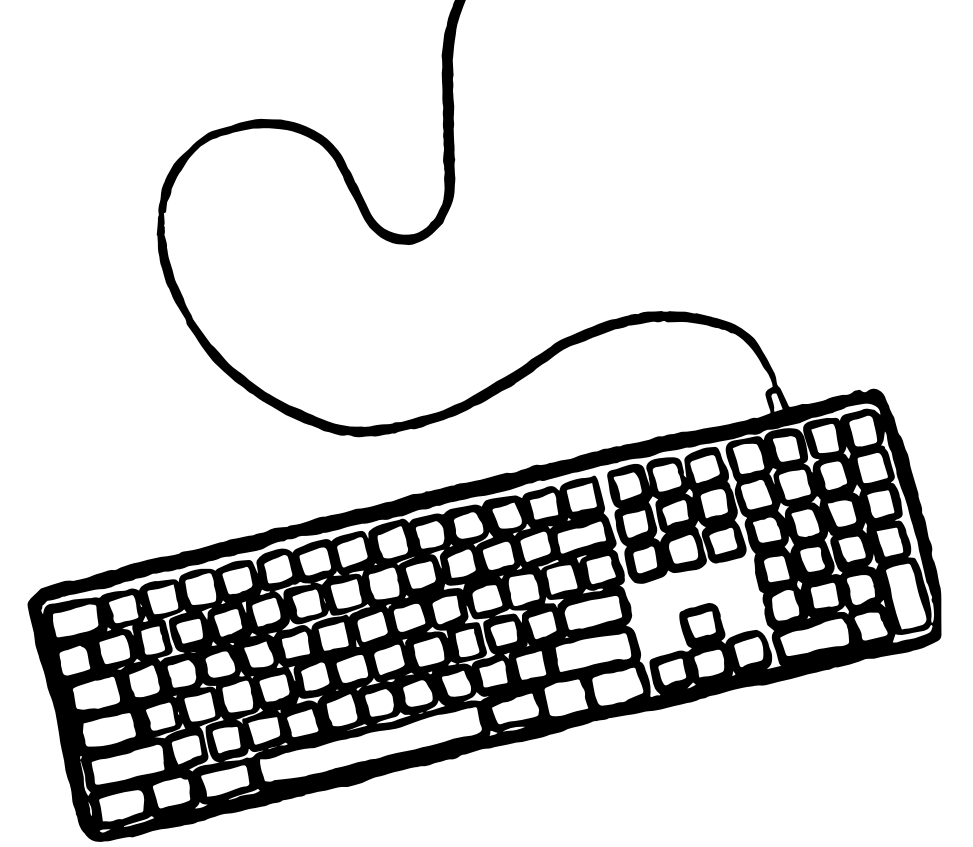


Enviando nossas alterações

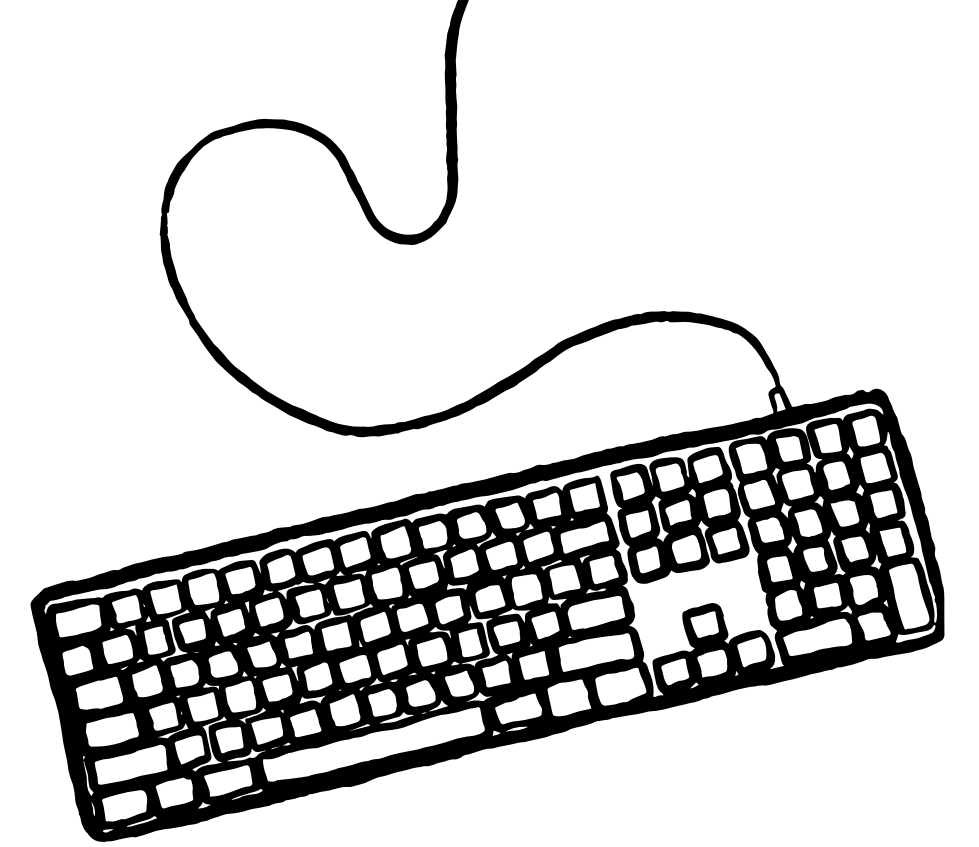
E para enviar as alterações ao repositório, devemos rodar um dos comandos abaixo.

```
git push --set-upstream origin feature/minhaBranch →  
ou  
git push
```

Apenas na 1ª vez que realizamos o comando na branch em que estamos



Guardando um trabalho inacabado



Em algum momento talvez você precise pausar seu trabalho e ajudar um colega em outra branch, mas não pode ou não quer commitar suas alterações

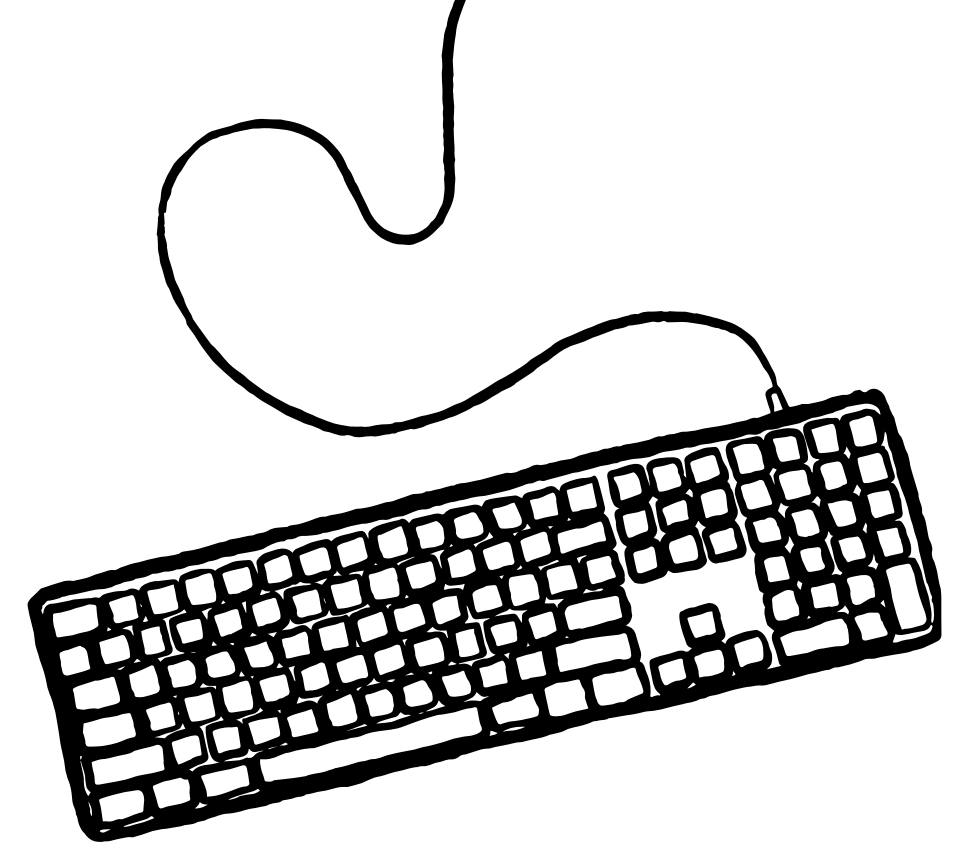
É possível guardar sua alteração em uma “gaveta temporária” com o comando ***git stash***

Depois, você pode voltar à sua branch e tirar o código da “gaveta” para retomar o trabalho

Guardando um trabalho inacabado

Na sua branch, edite `src/index.html` e adicione mais um elemento `<li>`

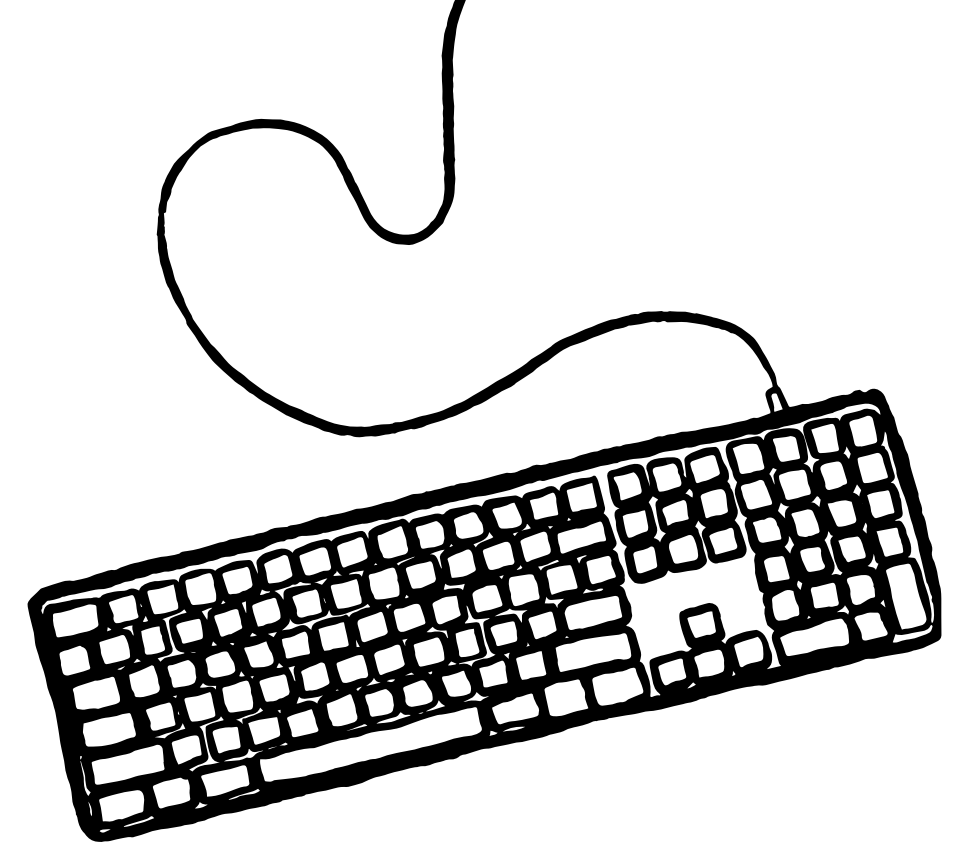
```
<li>SEU NOME</li>  
<li>Mais um elemento</li>
```



Guardando um trabalho inacabado

Dê CTRL + S para salvar o arquivo e rode o comando abaixo para colocar seu trabalho na “gaveta temporária”

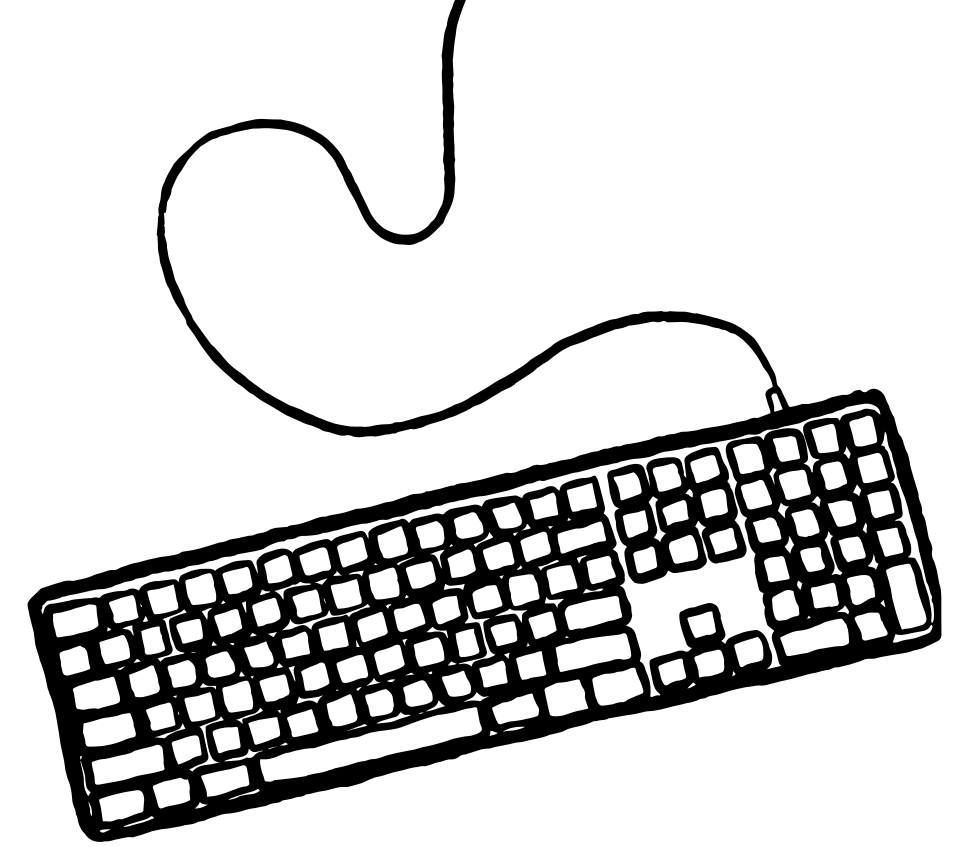
```
git stash
```



Guardando um trabalho inacabado

Agora vá para a branch main (principal) com o comando abaixo:

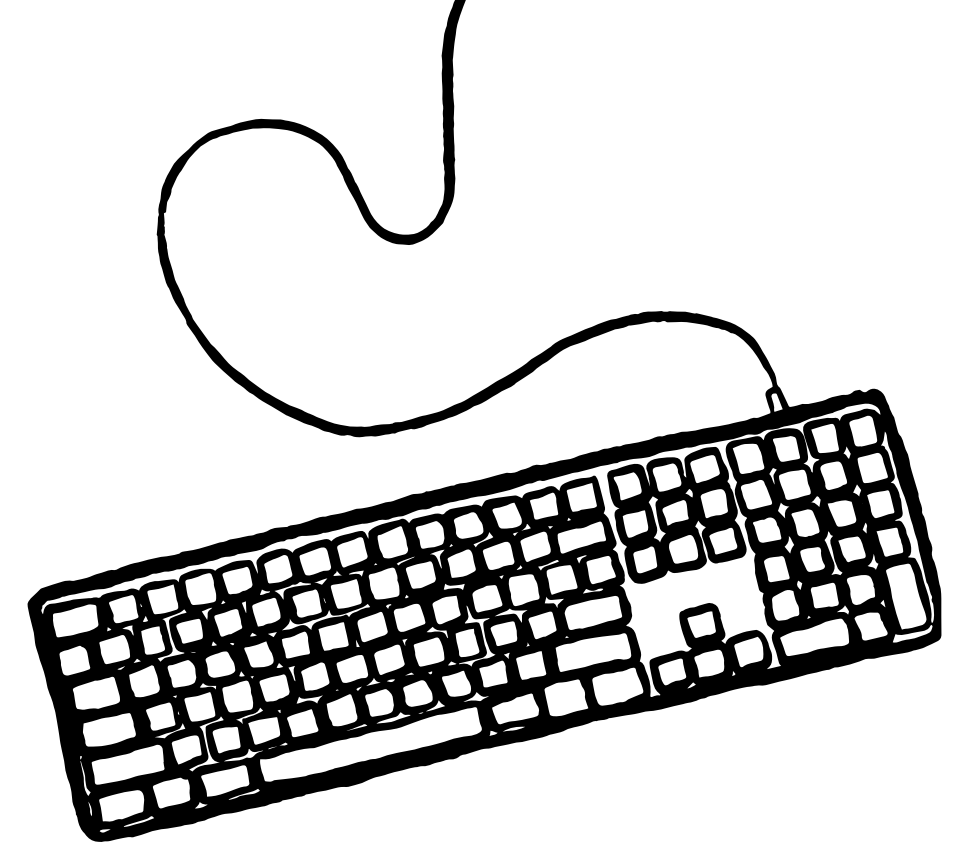
```
git checkout main
```



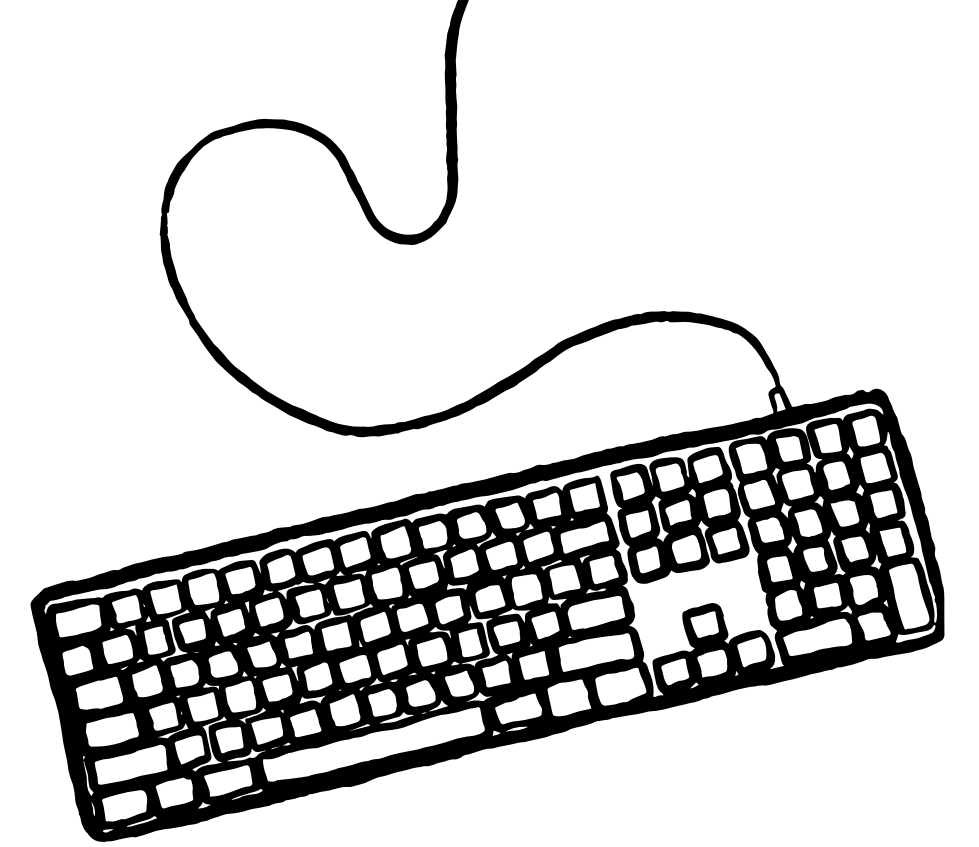
Guardando um trabalho inacabado

Supondo que você já viu o que precisava na branch main, retorne para a sua branch:

```
git checkout feature/<nome-feat>
```



Guardando um trabalho inacabado



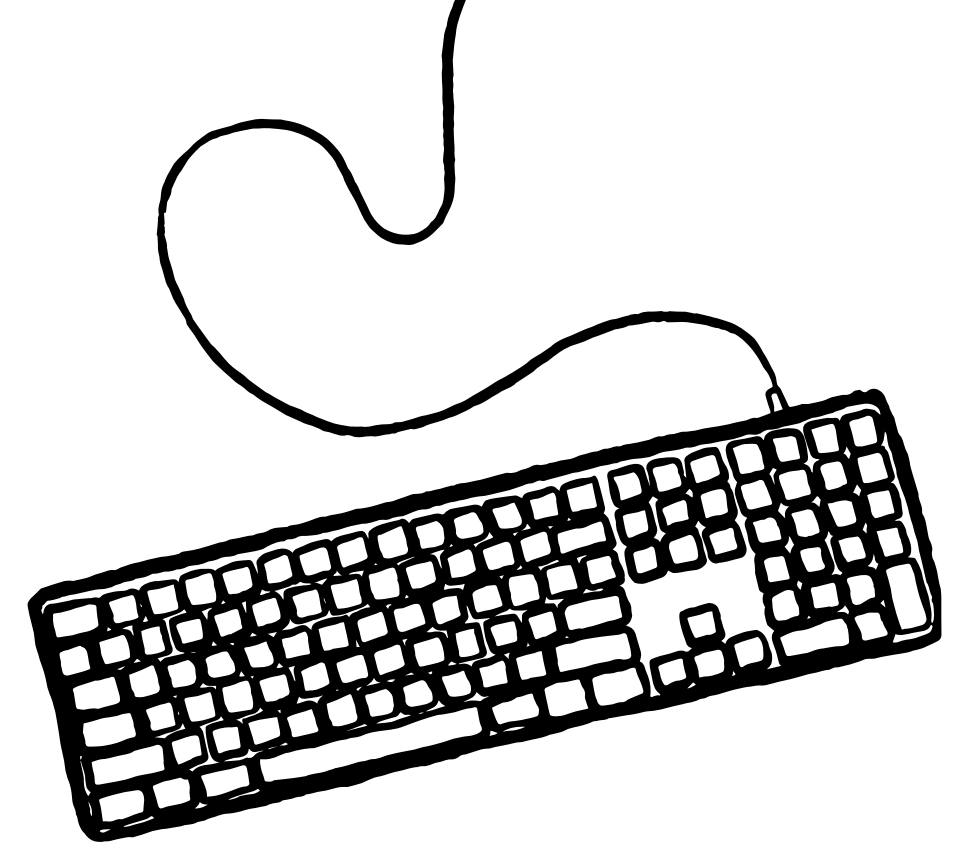
Lembre-se de sempre manter seu código atualizado com a versão da branch principal. Para fazer isso, rode o comando abaixo:

```
git pull
```

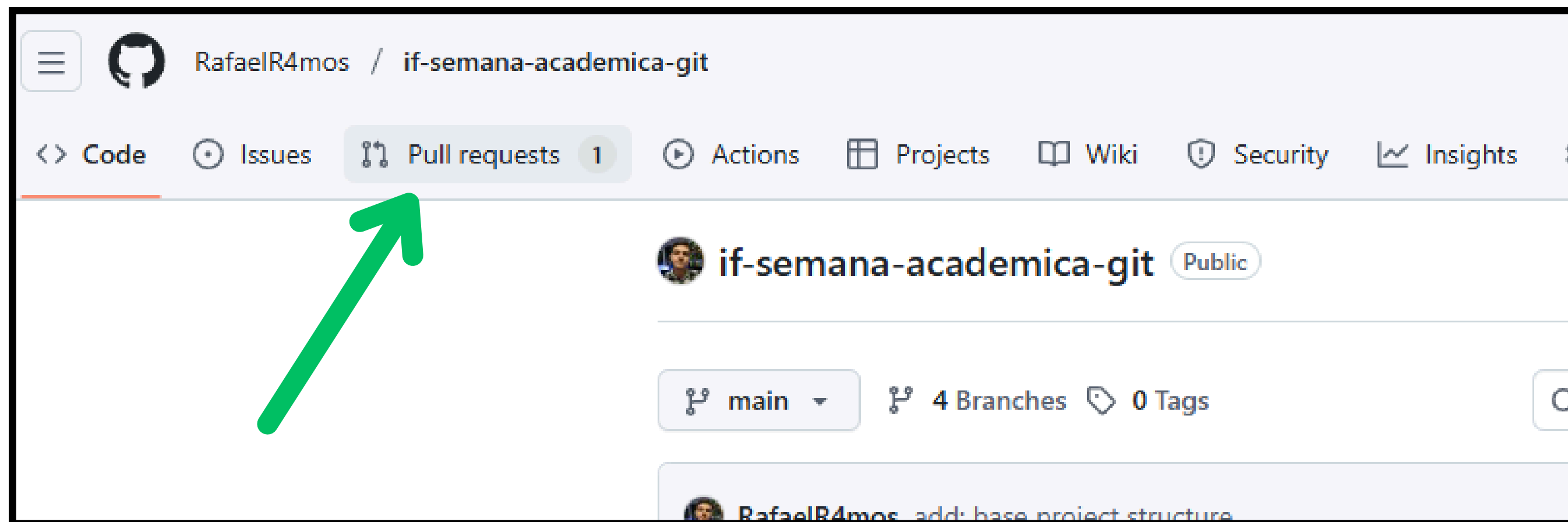
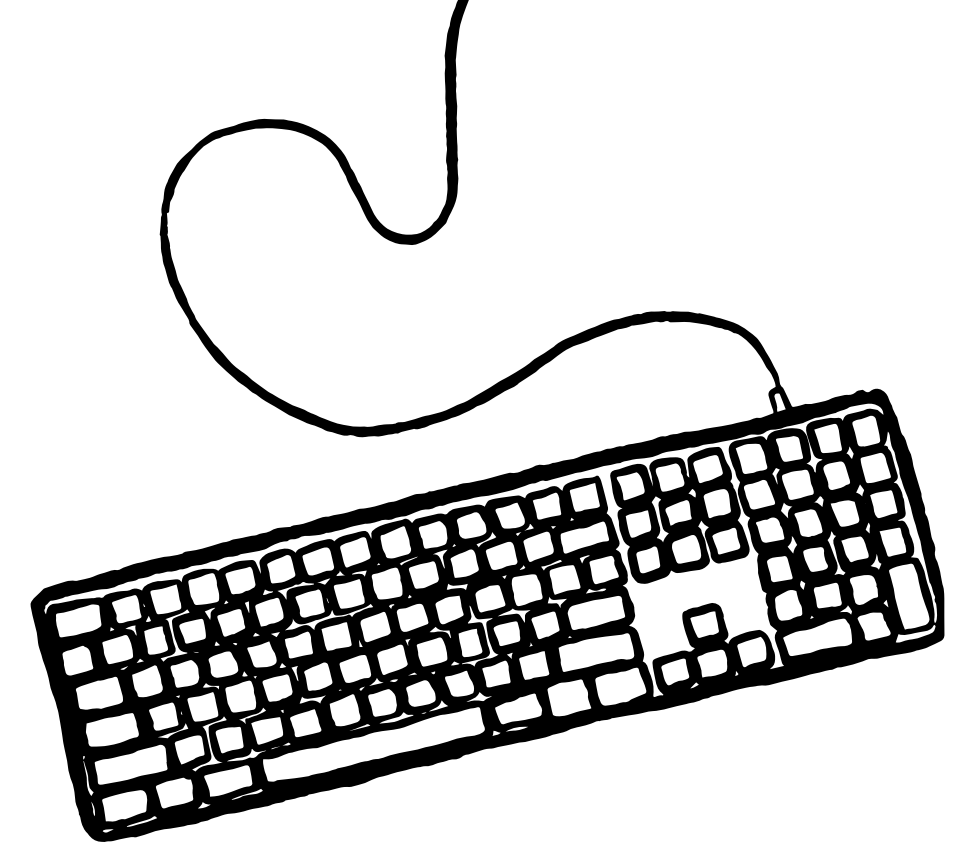
Guardando um trabalho inacabado

Por fim, rode o comando abaixo para reaplicar suas alterações:

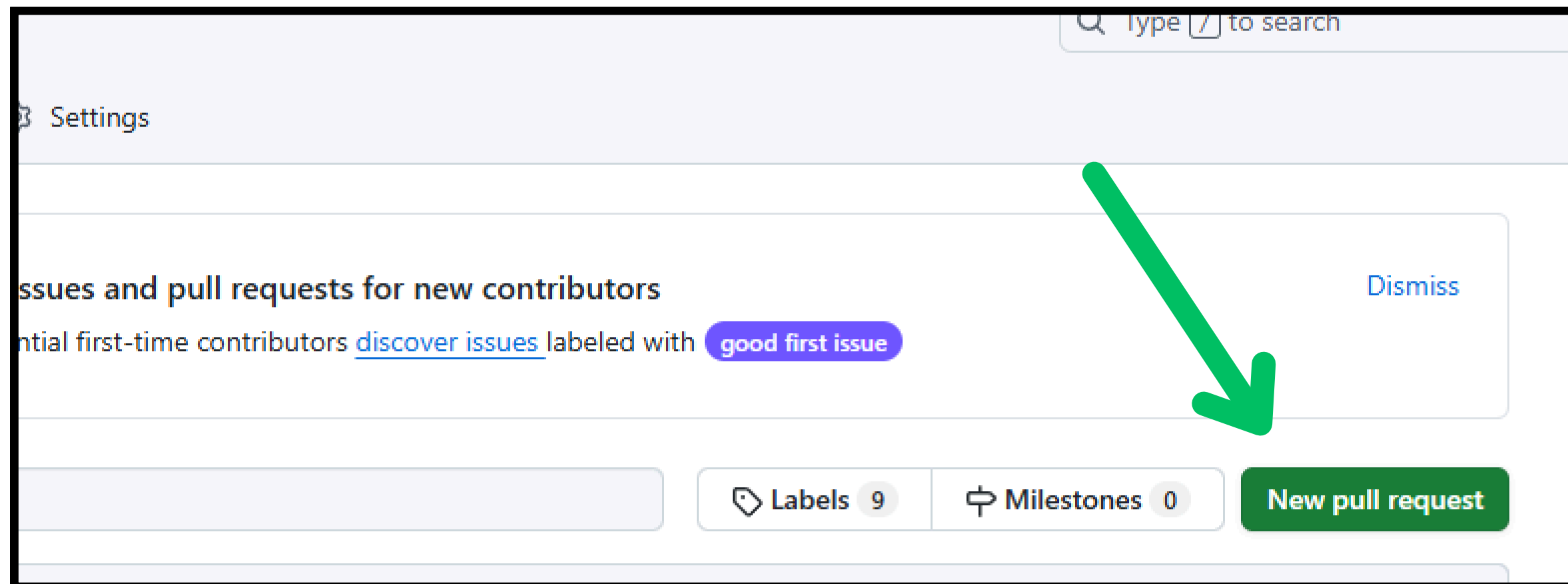
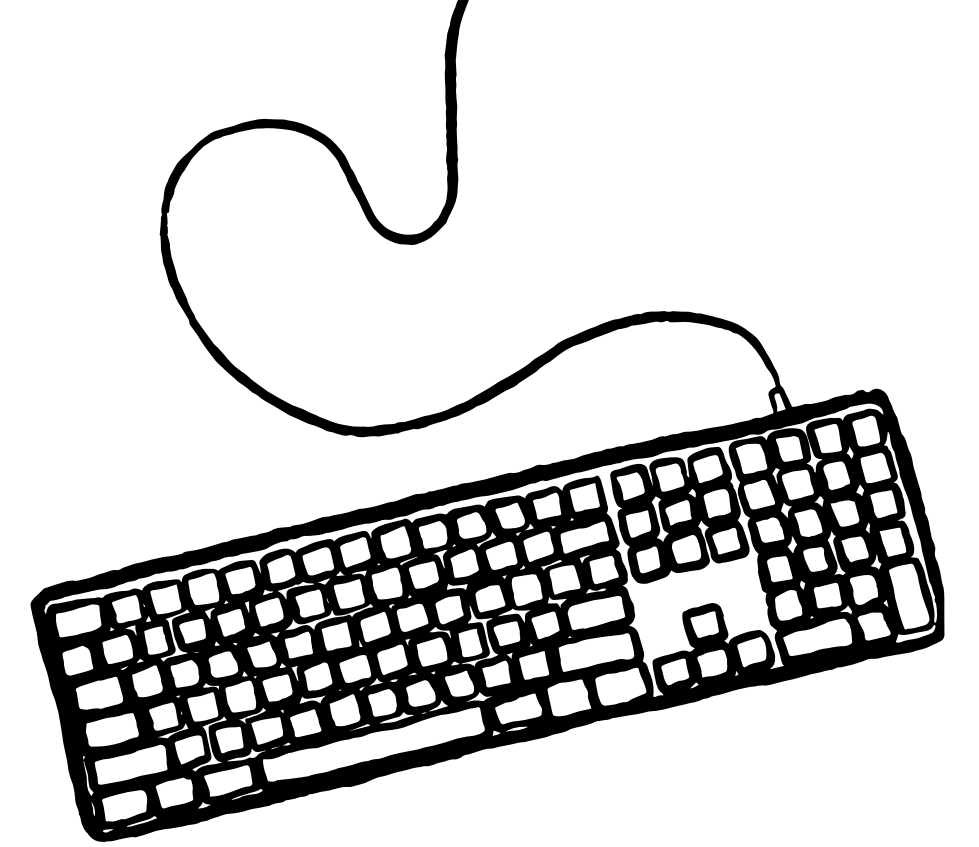
```
git stash apply
```



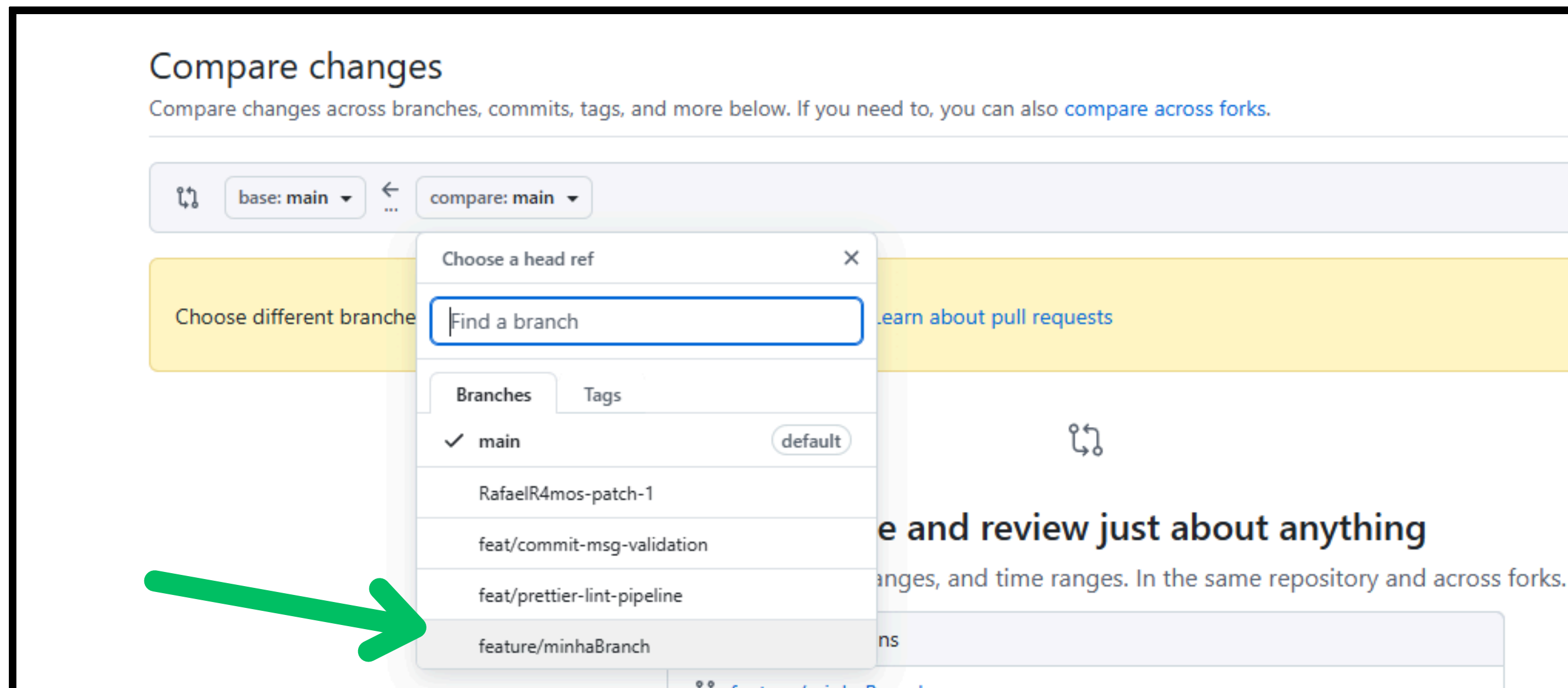
Abrindo um pull request



Abrindo um pull request



Abrindo um pull request



Abrindo um pull request

RafaelR4mos / if-semana-academica-git

Code Issues Pull requests 1 Actions Projects Wiki Security Insights Settings

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main compare: feature/minhaBranch ✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) **Create pull request**

1 commit 1 file changed 1 contributor

Commits on Oct 20, 2025

feat: removendo SEU NOME
ccarolb committed 2 minutes ago

Showing 1 changed file with 1 addition and 1 deletion.

src/index.html

@@ -25,7 +25,7 @@ Semana Acadêmica	
25 Carol Borges	25 Carol Borges
26 Felipe Selau	26 Felipe Selau
27 Rafael Ramos	27 Rafael Ramos
28 - SEU NOME	28 +
29	29
30 </body>	30 </body>
31 </html>	31 </html>

Abrindo um pull request: solicitando code review

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main

←

compare: feature/minhaBranch

✓ Able to merge. These branches can be automatically merged.

Add a title

feat: removendo SEU NOME

Add a description

WritePreview

H B I ≡ <> 🔗 ≡ ≡ 🔊 📎 @ ↻ ↺

Descrição

Removido o texto "SEU NOME"

Tipo de mudança

- [X] feat
- [] fix
- [] docs
- [] refactor
- [] test
- [] chore

Checklist

- [X] Branch nomeada corretamente
- [X] Commits seguem o padrão (Conventional Commits)
- [] Documentação atualizada (se necessário)
- [] Tests/linters ok (se houver)

Reviewers

Request up to 15 reviewers

felipeselau Felipe Scheffer

RafaelR4mos Rafael Ramos

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

A green arrow pointing from the right side of the screen towards the 'Reviewers' dropdown menu, specifically highlighting the list of suggested reviewers.

28

Abrindo um pull request: da minha branch para a main

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main

←

compare: feature/minhaBranch

✓ Able to merge. These branches can be automatically merged.

Add a title

feat: removendo SEU NOME

Add a description

WritePreview

H B I ≡ <> 🔗 ≡ ≡ 🔊 📎 @ ↻ ↺

Descrição

Removido o texto "SEU NOME"

Tipo de mudança

- [X] feat
- [] fix
- [] docs
- [] refactor
- [] test
- [] chore

Checklist

- [X] Branch nomeada corretamente
- [X] Commits seguem o padrão (Conventional Commits)
- [] Documentação atualizada (se necessário)
- [] Tests/linters ok (se houver)

Reviewers

Request up to 15 reviewers

felipeselau Felipe Scheffer

RafaelR4mos Rafael Ramos

Projects

None yet

Milestone

No milestone

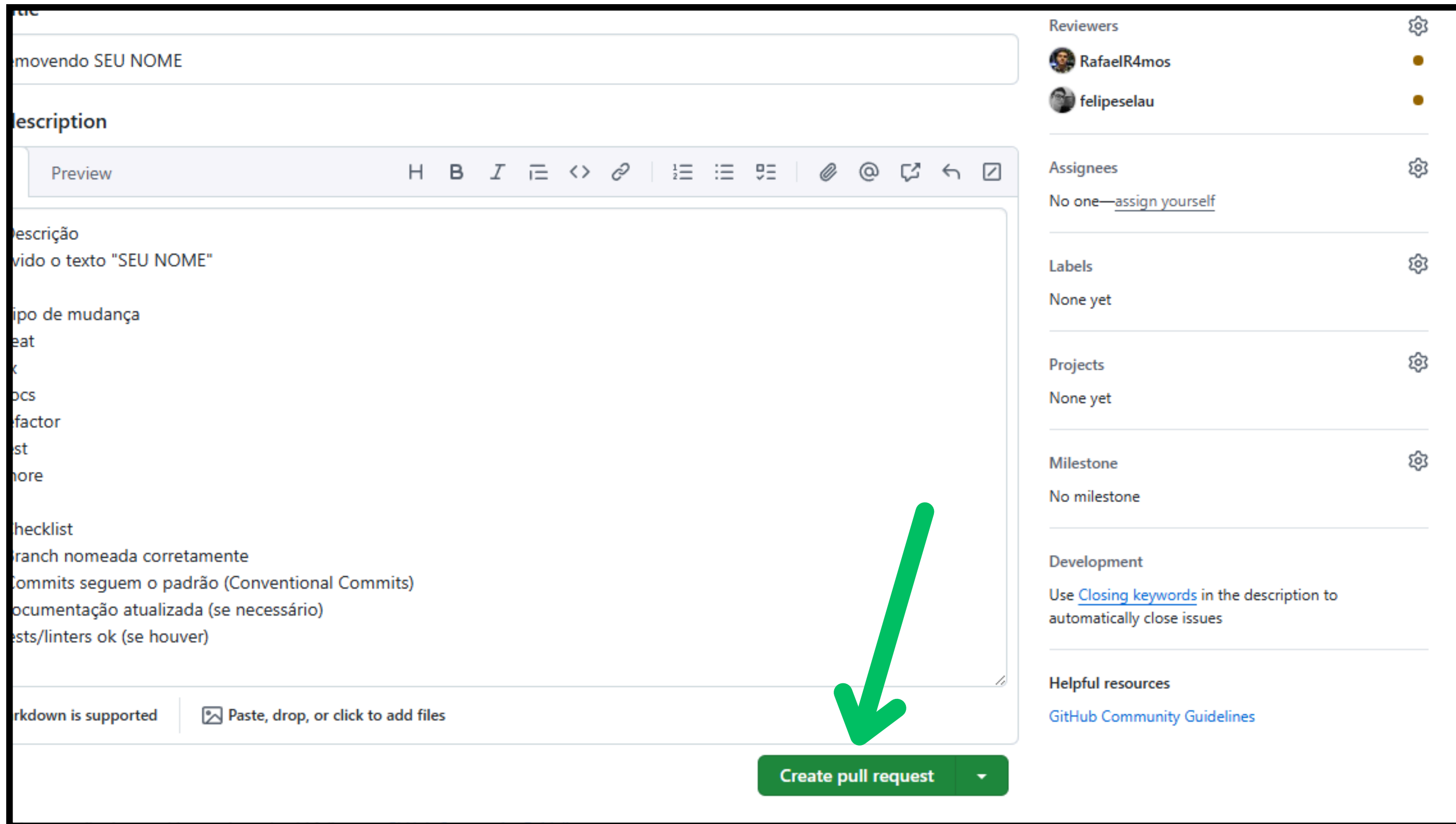
Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

Abrindo um pull request



The screenshot shows the GitHub interface for creating a pull request. The main area contains a text input field with the placeholder text "movendo SEU NOME". Below this is a "Description" section with a rich text editor toolbar (including bold, italic, link, and other icons) and a text area containing the following text:

Descrição
Mudou o texto "SEU NOME"

tipo de mudança
feat
fix
docs
refactor
test
chore

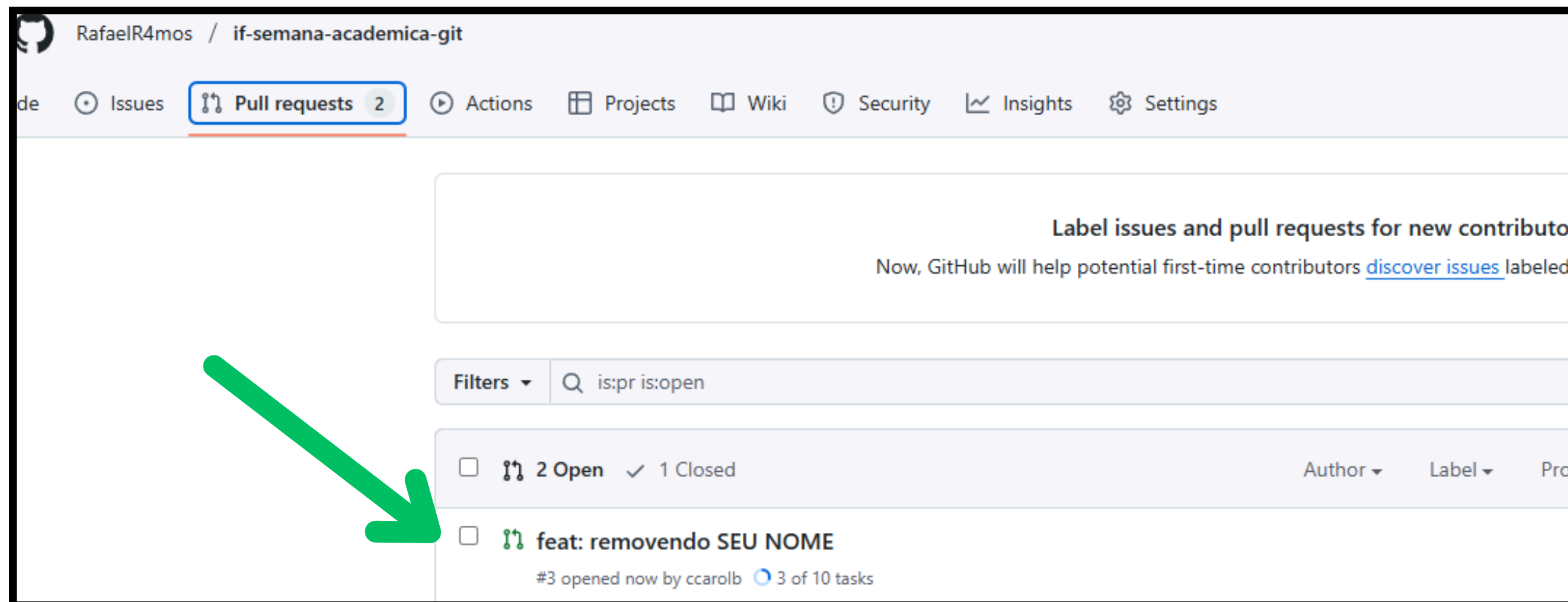
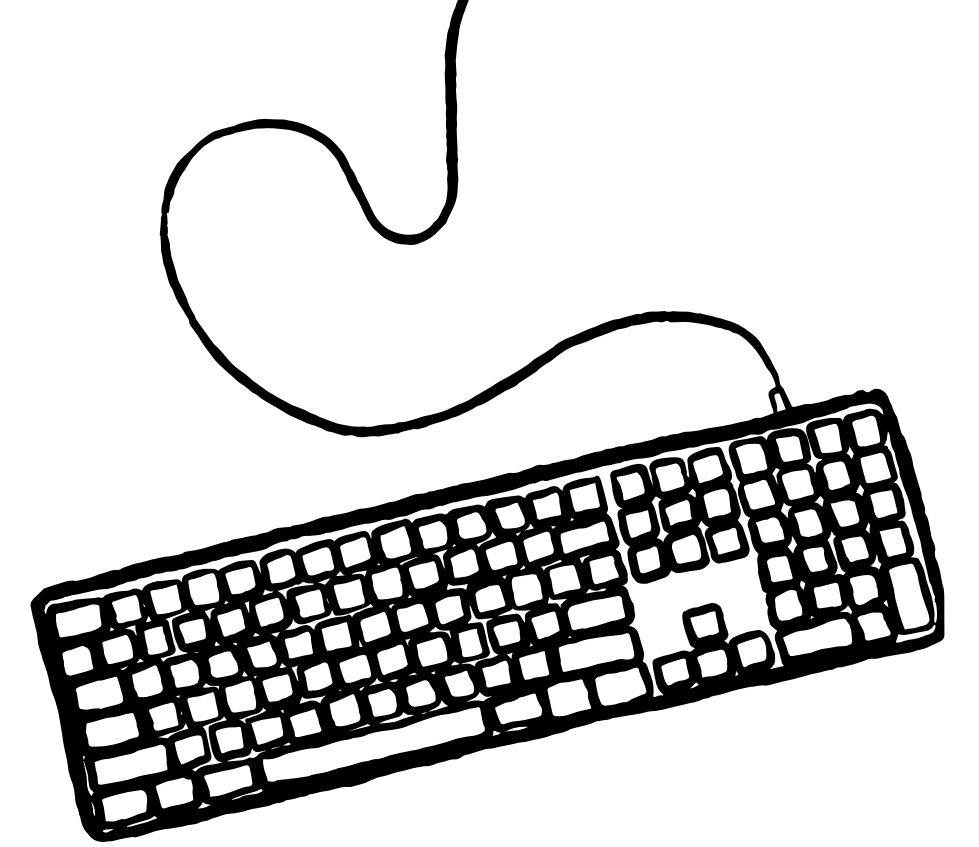
checklist
branch nomeada corretamente
commits seguem o padrão (Conventional Commits)
documentação atualizada (se necessário)
tests/linters ok (se houver)

At the bottom of the main area, there is a green button labeled "Create pull request" with a dropdown arrow. A large green arrow points to this button.

On the right side, there are several sections:

- Reviewers:** Lists "RafaelR4mos" and "felipeselau" with status indicators.
- Assignees:** Shows "No one" with a link to "assign yourself".
- Labels:** Shows "None yet".
- Projects:** Shows "None yet".
- Milestone:** Shows "No milestone".
- Development:** Includes a note: "Use [Closing keywords](#) in the description to automatically close issues".
- Helpful resources:** Includes a link to "GitHub Community Guidelines".

PR aberto, aguardando code review



Boas práticas: padrões de branches

feat/<nome-feat> → Nome de branch que representa a criação de uma nova funcionalidade

fix/<nome-fix> → Branch para correção de bugs ou problemas

docs/<nome-docs> → Branch para alterações na documentação

chore/<nome-chore> → Branch para tarefas de manutenção, atualizações de dependências, etc.

refactor/<nome-refactor> → Branch para refatoração de código sem alterar sua funcionalidade

test/<nome-test> → Branch para adição ou modificação de testes

style/<nome-style> → Branch para mudanças que não afetam o significado do código (formatação, espaços, etc.)

perf/<nome-perf> → Branch para mudanças de código que melhoram o desempenho

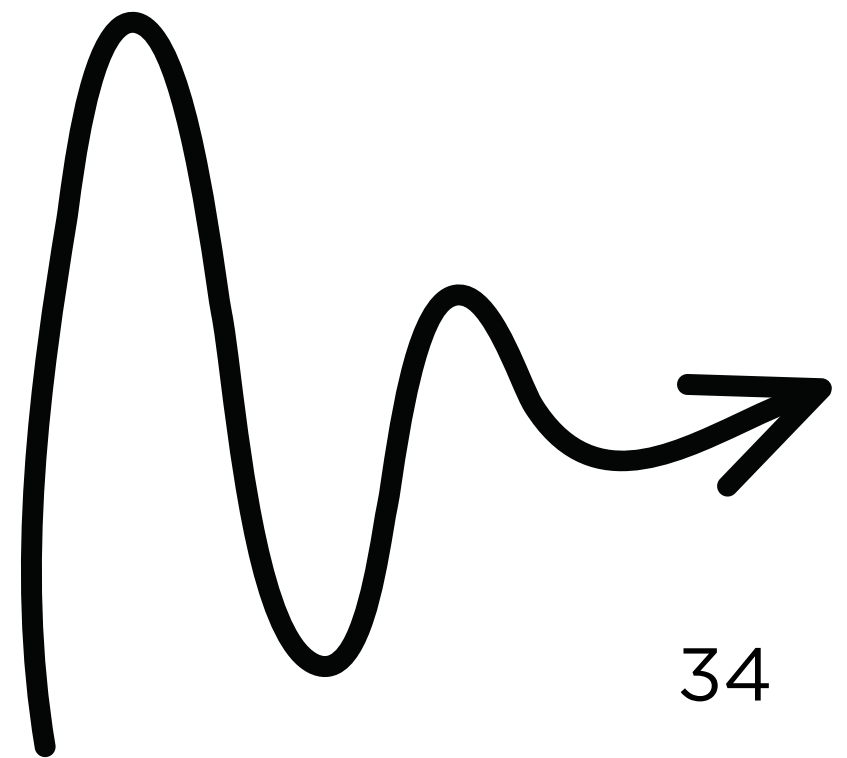


Boas práticas: padrões de commits

- **feat:** ... → Adiciona nova funcionalidade
- **fix:** ... → Corrige um bug
- **docs:** ... → Alterações na documentação
- **style:** ... → Mudanças que não afetam o significado do código
- **refactor:** ... → Refatoração do código
- **test:** ... → Adição ou modificação de testes
- **chore:** ... → Atualizações de tarefas de build, configurações, etc.
- **perf:** ... → Melhorias de performance



**Mas com tudo o que aprendemos,
como resolver conflitos?**





Hands-on: mãos na massa!



Obrigado!

LinkedIn dos palestrantes:

<https://www.linkedin.com/in/ccarolb/>

<https://www.linkedin.com/in/rafaelr4mos/>

<https://www.linkedin.com/in/luiz-felipe-selau/>

